1-5 인터페이스설계서

04 API: NOMAD Book Trailer (book-trailer)

products/book-trailer/04_API.md

작성일 2026-10-15. DB 함수는 `docs/products/_supabase-modular/migrations/2600_trailer.sql`이 정본이다. 코어 RPC는 `docs/products/_platform-core/CORE_SPEC.md`를 인용만 한다.

1. 원칙

항목	규칙
인증	모든 앱 Route Handler는 `requireSession('trailer', { roles })`. 조직은 세션에서만(요청 본문의 `org_id`를 믿지 않는다)
순서	인증 → 입력 검증(zod) → 플랜·권리 사전 검사(사용자에게 친절한 오류) → 크레딧 계산(`pricing.ts`) → service RPC(검사·선차감·enqueue 한 트랜잭션) → 202 응답
멱등	생성·렌더 요청은 클라이언트가 `request_id`(uuid v4)를 보낸다. 실행은 `run:trailer:<request_id>`, 렌더는 `renders.request_id`. 같은 값으로 다시 보내면 같은 결과를 돌려준다
선차감 위치	DB 함수 `start_run` / `start_render` 안에서만. Route Handler는 차감 RPC를 직접 부르지 않는다
응답 봉투	성공 `{ "data": ... }`, 실패 `{ "error": { "code": "TR005", "message": "사람이 읽는 문장", "detail": {...} } }`. 예외 원문·스택을 내보내지 않는다
진행 상황	Realtime 채널 `trailer:<org>:runs`, `trailer:<org>:renders`(행 변경 구독). 폴링은 대체 수단(5초)
레이트 리밋	인증 API는 `SECURITY_BASELINE.md` 8장 기본값(코어 `rate_limit_buckets`). 생성 API는 사용자당 분당 20, `/b/[code]` IP당 분당 60, `/review/[token]` IP당 분당 20(제안)
업로드	서명 업로드 URL 발급 → 클라이언트가 Storage에 직접 업로드 → 완료 통지 → 서버가 크기·MIME·매직바이트 재검사. 경로는 서버가 만든다

2. Route Handler

메서드 경로	역할/플랜	입력(zod 요약)	출력	크레딧	호출 RPC	오류
`POST /api/books`	member+	`{ title, subtitle?, author_name?, publisher_name?, genre, language, isbn?, published_on?, description?, audience?, source: 'upload'|'paste' }`	book	0	INSERT(RLS)	`TR005`, 23514
`PATCH /api/books/:id`	member+	서지 필드 일부, `status`	book	0	UPDATE(RLS)	23514
`DELETE /api/books/:id`	owner/admin	-	`{ deleted_at }`	0	UPDATE `deleted_at` → 30일 뒤 purge	-

메서드 경로	역할/플랜	입력(zod 요약)	출력	크레딧	호출 RPC	오류
`POST /api/books/:id/rights`	member+	`{ version: 'rights-2026-10-v1' }`	book	0	`trailer.acknowledge_rights`	22023
`POST /api/uploads/sign`	member+	`{ book_id, kind: 'manuscript'|'cover'|'music'|'scene_image'|'logo', ext, bytes }`	`{ path, signed_url, expires_at }`	0	Storage 서명 업로드	413(상한 초과), 415(확장자)
`POST /api/uploads/complete`	member+	`{ path, kind, scene_id?, rights_ack? }`	대상 행	0	매직바이트 검사 → 원고면 `book.parse` 잡, 표지면 `kit.extract` 잡, 장면 이미지면 `asset.normalize` 잡 (service enqueue)	415, 422(권리 확인 누락)
`POST /api/books/:id/paste`	member+	`{ text }` (600,000자)	book	0	md로 Storage 저장 → `book.parse`	413
`POST /api/books/:id/digest`	member+	`{ request_id }`	run	20 + 10만 자당 8	`trailer.start_run('digest')`	`TR003`, `TR004`, `TR002`, `CR001` (402), 22023(이미 분석됨)
`GET /api/books/:id/links` / `POST` / `PATCH /api/links/:id` / `DELETE`	member+	`{ store, label?, url, is_affiliate, sort_order? }`	link(+ `short_url`)	0	RLS	`TR005` (11번째), 23514(http)
`GET /api/books/:id/stats`	member+	`?period=7d|30d`	링크·태그별 클릭	0	SELECT `link_clicks`	-
`POST /api/brand-kits` / `PATCH /api/brand-kits/:id`	member+	`{ book_id?, name, palette[], font_display, font_body, logo_path? }`	kit	0	RLS	23514(팔레트), `TR005`
`POST /api/trailers`	member+	`{ book_id, title, grammar_code, template_code, formats[], duration_s, language, spoiler_level, narration_mode, voice_pref, brand_kit_id? }`	trailer(+엔드카드)	0	INSERT(RLS)	`TR005`, 23503
`PATCH /api/trailers/:id`	member+	설정 필드, `current_brief_id`, `selected_hook`, `music_*`, `status`	trailer	0	UPDATE(RLS)	`TR004` (승인 조건), 23503
`POST /api/trailers/:id/brief`	member+	`{ request_id }`	run	8	`start_run('brief')`	`TR004` (분석 전)

메서드 경로	역할/플랜	입력(zod 요약)	출력	크레딧	호출 RPC	오류
`POST /api/trailers/:id/briefs`	member+	`{ content }` (4.2절)	brief(manual)	0	INSERT(RLS)	23514(64KB)
`POST /api/trailers/:id/storyboard`	member+	`{ request_id }`	run	6	`start_run('storyboard')`	`TR004` (브리프 미선택)
`PATCH /api/scenes/:id`	member+	`{ patch: {text?, narration?, duration_ms?, visual?, music_cue?}, revision }`	`{ revision }`	0	`trailer.save_scene`	`TR409`, 22023
`POST /api/trailers/:id/scenes` / `DELETE /api/scenes/:id`	member+	장면 필드	scene	0	RLS(엔드카드 제외)	42501(엔드카드)
`PUT /api/trailers/:id/scene-order`	member+	`{ scene_ids[] }`	204	0	`trailer.reorder_scenes`	`TR007`, 22023
`POST /api/scenes/:id/image`	member+	`{ request_id }`	run	3	`start_run('scene_image')`	22023(종류)
`POST /api/scenes/:id/narration`	member+	`{ request_id }`	run	EL 2 / OSS 1	`start_run('narration')`	`TR004` (문구 없음)
`POST /api/trailers/:id/narration/reset`	member+	`{ request_id, voice_pref }`	runs[]	장면 수 x 단가	서버(service)가 `voice` 비우고 audio superseded → 장면마다 `start_run('narration')`	`CR001`
`POST /api/trailers/:id/meta`	member+	`{ request_id, platforms[] }` (4개 이하)	run	2	`start_run('meta')`	`TR004`
`PATCH /api/meta/:id`	member+	`{ title?, description?, hashtags?, link_ids? }`	pack	0	UPDATE(RLS, 트리거가 대가성 문구)	23514, 23503
`POST /api/trailers/:id/renders`	member+	`{ request_id, format, resolution, confirm_unverified_quotes?, confirm_warnings? }`	render	4~10	서버 compliance 사전 검사 → `trailer.start_render`	`TR003`~`TR007`, `TR011`, `CR001`, 409(WARN 미확인)
`GET /api/renders/:id/download`	member+	`?part=video|poster|srt|zip`	302 서명 URL(10분)	0	SELECT + Storage	410(만료)

메서드 경로	역할/플랜	입력(zod 요약)	출력	크레딧	호출 RPC	오류
`POST /api/renders/:id/rerender`	member+	`{ request_id }`	render	렌더 단가	같은 스냅샷으로 `start_render` (만료 재생성)	-
`POST /api/reviews` (P1)	member+, publisher	`{ trailer_id, render_ids[], reviewer_label }`	`{ review, url }` (토큰 원문은 이 응답에서 한 번만)	0	`trailer.open_review` (sha256만 저장)	`TR005`
`POST /api/import/flora` (P1)	member+, creator+	`{ flora_book_id }`	book	0	`trailer.import_flora_book` → 원고 md 내보내기 → `book.parse`	`TR010`, `TR005`
`POST /api/channels/youtube/connect` (P1)	owner/admin	-	OAuth URL	0	코어 `create_oauth_state`	-
`POST /api/renders/:id/upload` (P1)	member+, creator+	`{ channel_id, metadata_pack_id, publish_at? }`	upload	1	enqueue `upload.youtube`	경고 409(같은 템플릿 반복)
`GET /b/:code`	공개	`?t=<태그>`	302 원 링크	0	`trailer.record_link_click` (service)	404, 429
`GET /review/:token`, `POST /review/:token` (P1)	공개(토큰)	`{ decision, comment? }`	승인 화면 / 결과	0	서버가 sha256 → `trailer.decide_review`	`TR009` (410), 429
`POST /api/webhooks/toss`	공개(서명)	Toss 이벤트	200	-	코어 웹훅 처리	401(서명)

2.1 AI·음성 Route Handler

경로	입력	출력	크레딧	서명 토큰 scope
`POST /api/ai/conversations`	`{ book_id }`	`{ conversation_id }`	0	-
`POST /api/ai/chat`	`{ conversation_id, message }`	SSE 스트림(출처: 장 번호·문단 위치)	선차감 3, 정산 1~5	`chat`
`POST /api/voice/consent`	`{ version }`	204	0	-
`POST /api/voice/stt`	음성(60초, 5MB) + `{ feature: 'voice_chat'|'dictation' }`	`{ text }`	분당 1(오픈소스) / 3(EL)	`voice`
`POST /api/voice/tts`	`{ message_id }` (서버가 가진 답변 텍스트만, 임의 텍스트 불가)	오디오 스트림	천 자당 1 / 5	`voice`
`POST /api/voice/sample`	`{ voice_pref, voice_id }` (100자 고정 문장)	오디오	1	`voice`
`GET /api/wiki?book_id=` / `PATCH /api/wiki/:page_id`	위키 페이지	페이지	0 / 0	`wiki`
`POST /api/wiki/lint`	`{ book_id }`	run	1	`wiki`
`POST /api/internal/ai-token`	내부 키 + `{ job_id }`	잡 토큰(running 잡의 org·user, 10분)	-	`graph:trailer.*`, `voice`, `parse`

경로	입력	출력	크레딧	서명 토큰 scope
`POST /api/internal/ai-settle`	내부 키	정산 건수	-	-
`POST /api/onboarding/cross-border`	`{ ack: boolean }`	204	0	코어 `set_ai_cross_border_ack`

3. RPC (SECURITY DEFINER / INVOKER)

시그니처	호출자	검사	반환	SQLSTATE
`trailer.acknowledge_rights(p_book_id uuid, p_version text)` DEFINER	authenticated	owner/admin/member, 엔타이틀먼트, 버전 형식. 감사 로그	books	P0002, 42501, 22023
`trailer.save_scene(p_scene_id uuid, p_patch jsonb, p_expected_revision int)` INVOKER	authenticated	RLS + 컬럼 GRANT, 키 허용 목록	새 revision	`TR409`, P0002, 22023
`trailer.reorder_scenes(p_trailer_id uuid, p_scene_ids uuid[])` INVOKER	authenticated	전체 id 일치, 엔드카드 마지막	void	`TR007`, 22023, P0002
`trailer.plan_features(uuid)`, `plan_code(uuid)`, `has_trailer_access()`, `feature_int/bool/has`, `valid_palette`, `valid_hashtags`	authenticated	멤버 또는 service	jsonb/text/bool	42501
`trailer.start_run(p_org_id, p_kind, p_book_id, p_trailer_id, p_scene_id, p_credits, p_idempotency_key, p_actor, p_payload)`	service_role	키 형식 `run:trailer:<rid>`, 멱등·조직 일치, 행위자 멤버십, 엔타이틀먼트, 대상·부모 일치, 권리 확인, 선행 단계, 같은 대상 진행 중, 조직 동시 실행 4, 선차감, enqueue(`trailer.ai`)	generation_runs	42501, 22023, P0002, `TR001` ~ `TR005`, `CR001`
`trailer.finish_run(p_run_id, p_result jsonb, p_ai_run_id)`	service_role	실행 활성, kind별 결과 형식(4.2~4.3절), 엔드카드 미포함, 같은 목소리	generation_runs	`TR001`, `TR007`, 22023, 23514
`trailer.fail_run(p_run_id, p_error)`	service_role	활성 실행	generation_runs	`TR001`
`trailer.reconcile_runs(p_org_id)`	service_role	잡이 끝난 실행·렌더를 failed/canceled로	정리 건수	-

시그니처	호출자	검사	반환	SQLSTATE
`trailer.start_render(p_org_id, p_trailer_id, p_format, p_resolution, p_credits, p_request_id, p_actor, p_allow_unverified_quotes)`	service_role	멱등, 멤버십, 엔타이틀먼트, 보관 여부, 권리, 포맷(트레일러 선택 + 플랜), 해상도(플랜), 엔드카드 마지막, 장면 2개 이상, 길이 한도, 인용 검증·한도, 에셋 준비, 음악 권리·공개, 템플릿 지원·등급, 동시 렌더 2, 워터마크·AI 표시 결정, 스냅샷, 선차감, enqueue(`trailer.render` , 우선순위)	renders	42501, 22023, P0002, `TR001` , `TR003` ~ `TR007` , `TR011` , `CR001`
`trailer.finish_render(p_render_id, p_output_path, p_poster_path, p_srt_path, p_bytes, p_duration_ms)`	service_role	활성, 경로가 그 렌더 폴더, 보존 기간	renders	`TR001` , 22023, P0002
`trailer.fail_render(p_render_id, p_error)`	service_role	활성	renders	`TR001`
`trailer.record_link_click(p_code, p_referrer, p_ip_hash, p_user_agent, p_tag)`	service_role	활성 링크·삭제되지 않은 책	원 URL 또는 null	-
`trailer.open_review(p_org_id, p_trailer_id, p_render_ids, p_token_sha256, p_label, p_actor)`	service_role	멤버, `approval_links` , 같은 트레일러의 성공 렌더	reviews	`TR005` , 22023, P0002
`trailer.decide_review(p_token_sha256, p_decision, p_comment)`	service_role	pending·미만료	reviews	`TR009` , 22023
`trailer.import_flora_book(p_org_id, p_flora_book_id, p_actor)`	service_role	publishing 스키마 존재, 두 엔타이틀먼트, `flora_import` , 같은 조직 책	books	`TR010` , `TR005` , P0002

코어 RPC(인용만): `consume_credits` , `refund_credits` , `enqueue_job` , `claim_jobs` , `complete_job` , `fail_job` , `cancel_job` , `write_audit` , `resolve_ai_route` , `set_ai_cross_border_ack` , `match_ai_chunks` , `create_oauth_state` , `confirm_payment_order` , `fail_payment_order` .

4. 잡 페이로드 계약

TS(zod)와 Python(pydantic) 양쪽에서 같은 스키마로 검증한다. 원본 스키마 파일은 `packages/trailer-compositions/src/schema/` (TS)와 `apps/api/app/ai/products/trailer/schema.py` (Python), 둘의 동치 테스트가 CI에 있다(TR-06-024).

4.1 잡 payload / result / progress

job_type	payload	result(워커 → `finish_*`)	progress(`generation_runs.progress`)
`book.parse`	`{ book_id, path }`	서버가 `books.manuscript_sha256/chars` 와 정규화 md 경로 갱신	-

job_type	payload	result(워커 → finish_*)	progress(generation_runs.progress)
trailer.digest	{ run_id, kind, book_id }	{ manuscript_sha256, chapters: [{n, title, chars, summary, key_passages:[{start,end}]}], characters: [{name, aliases[], description}], themes: [string], settings: [{name, description}] }	{ stage: 'split'|'summarize'|'entities'|'index'|'wiki', done, total }
trailer.brief	{ run_id, kind, book_id, trailer_id }	{ brief: 4.2절 }	{ stage }
trailer.storyboard	{ run_id, kind, trailer_id }	{ scenes: [4.3절 장면, 1~39개, 엔드카드 제외] }	{ stage }
trailer.scene_image	{ run_id, kind, trailer_id, scene_id }	{ storage_path, provider: 'fal', model, seed, width, height, bytes }	-
trailer.narration	{ run_id, kind, trailer_id, scene_id }	{ storage_path, provider, model, voice_id, duration_ms, chars, bytes }	-
trailer.meta	{ run_id, kind, trailer_id }	{ packs: [{ platform, title, description, hashtags[], link_ids[] }] }	-
render.trailer	{ render_id } (설정은 스냅샷에서만 읽는다)	finish_render(output_path, poster_path, srt_path, bytes, duration_ms)	core.jobs.progress 0~100(프레임 비율)
kit.extract	{ book_id, cover_path }	서버가 brand_kits 에 source='cover_extract' 행 생성	-
asset.normalize	{ scene_id, path, rights_ack: true }	서버가 scene_assets (user_image, upload) 행 생성	-

4.2 브리프 content

```
{
  "angle": "character | question | atmosphere | problem | insight",
  "logline": "60자 이하",
  "hooks": [{ "text": "40자 이하" }],
  "quotes": [{ "text": "80자 이하", "chapter": 3, "start": 1204, "end": 1236, "verified": true }],
  "audience": "100자 이하",
  "mood": "cinematic_tension | calm_piano | upbeat_focus | ...",
  "keywords": ["..."],
  "themes": ["..."],
  "spoiler_notes": ["결말 구간 인용 제외: 11~12장"],
  "claims": [{ "text": "베스트셀러", "evidence": null }],
  "compliance": { "verdict": "pass | warn | fail", "items": [] }
}
```

AI 실행 1회가 3안을 만들면 브리프 3행(각 angle)을 넣는다. finish_run 은 첫 행을 현재 브리프로 지정한다(이미 있으면 바꾸지 않는다).

4.3 장면(스토리보드 결과와 편집 단위)

```
{
  "kind": "title_card | hook_text | quote_card | image_scene | character_scene | narration_card | cta_card",
  "duration_ms": 3000,
  "text": "화면 문구 160자 이하",
  "narration": "내레이션 400자 이하",
```

```
  "quote_ref": { "chapter": 3, "start": 1204, "end": 1236 },
  "quote_verified": true,
  "visual": {
    "prompt": "영어 이미지 프롬프트 600자 이하",
    "style": "문법 스타일 프리셋 코드",
    "motion": "ken_burns_in | ken_burns_out | pan_left | pan_right | parallax | still | type_reveal | page_turn",
    "focus": "center | top | bottom | left | right"
  },
  "music_cue": "intro | build | drop | calm | outro"
}
```

4.4 렌더 스냅샷 → 컴포지션 props

`renders.snapshot`(DB가 만든다, `2600_trailer.sql` 7-5절) 키: `trailer`(id, title, duration_s, language, voice, storyboard_version), `book`(title, subtitle, author_name, publisher_name, cover_path, genre), `template`(code, composition_id, version), `brand_kit`(palette, font_display, font_body, logo_path 또는 null), `music`(track_id, path, storage_path), `cta`(short_code, store, label), `scenes`(장면 + `image`, `audio`), `total_ms`, `has_ai_assets`. 렌더러는 여기에 요청 시점 값 `format`, `resolution`, `watermark`, `ai_label`(renders 행)만 더해 props를 만든다. 브랜드 키트가 없으면 템플릿 기본 팔레트(08_DESIGN_SYSTEM.md 6장).

5. 웹훅

방향	제공자/이벤트	서명	event_id	처리
수신	Toss 결제	코어 규칙(서명 필수)	Toss 이벤트 id	코어 `webhook_events` 멱등 → 상태 동기화만
수신	fal(비동기 결과, 선택)	fal 서명 확인(방식 미확인, 확인 전에는 동기 호출 + 폴링만)	request id	오케스트레이터가 기다리는 실행에 결과 연결
발신	없음(MVP)	-	-	Phase 2에 출판사 연동(렌더 완료 알림) 검토

6. 외부 API 어댑터

외부	호출 위치	실패 처리	쿼터
fal(Flux)	오케스트레이터 `steps/image`	3회 지수 백오프(같은 seed), 안전 검사 차단은 재시도 없이 실패 + 환불 + "다른 장면 설명" 안내	조직당 동시 이미지 4(제안)
ElevenLabs	`core-voice`(Next.js 서버, 오케스트레이터)	같은 제공자로만 재시도(내레이션)	계정 동시 요청 한도(구현 시 확인)
공용 음성 서버	`core-voice`	같은 규칙	운영사 서버
YouTube Data API(P1)	공용 sns-uploader	쿼터 초과는 `run_after` 재예약	프로젝트 일일 쿼터
Toss	코어	코어 규칙	-

6.1 이 제품이 쓰는 FastAPI 엔드포인트

엔드포인트	호출자	인증	내용
`POST /products/trailer/parse`	오케스트레이터(`book.parse`)	잡 토큰 scope `parse`	Storage 원고 → 텍스트 추출 → 정규화 md 저장 → `{sha256, chars, md_path}`

엔드포인트	호출자	인증	내용
`POST /graphs/trailer.digest/runs`	오케스트레이터	잡 토큰 `graph:trailer.digest`	4.1절 result. `ai_runs` · `ai_run_steps` · `ai_usage` 기록
`POST /graphs/trailer.brief/runs`	오케스트레이터	잡 토큰	브리프 3안
`POST /graphs/trailer.storyboard/runs`	오케스트레이터	잡 토큰	장면 배열
`POST /graphs/trailer.meta/runs`	오케스트레이터	잡 토큰	메타 팩
`GET /graphs/runs/{id}`	오케스트레이터	잡 토큰	상태·결과(재시도 시 같은 실행 재개)
`POST /products/trailer/quotes/verify`	Next.js(사용자가 인용 장면을 원문으로 되돌릴 때)	서명 토큰 `chat`	`{ text }` → `{ verified, chapter, start, end }`. 맞으면 서버(service)가 `quote_verified=true`
`POST /tools/answer`	Dify	내부 키 + 대화 매핑	코치 답변(책 컬렉션·위키만)
`POST /rag/ingest`, `/wiki/*`	그래프 내부, Next.js	서명 토큰	공용 ai-service(`AI_ARCHITECTURE.md` 8장)

6.2 Dify Chatflow 계약 (`trailer-chat`)

항목	값
입력 변수	`platform_conversation_id`(필수). Dify `user` = Next.js가 만든 `dify_user`(HMAC)
답변	HTTP 노드 → `POST {FASTAPI}/tools/answer`(대화 → 조직·책을 서버가 찾는다)
보조 LLM 노드	`dify.aux`(오픈소스 강제). 조직 데이터 금지
응답	텍스트 + 출처 배열(`{chapter, start, end, snippet}`)
금지	Dify 내장 지식베이스, 토큰·원고·개인정보를 입력 변수로 넘기기

7. 오류 코드

코드	의미	HTTP
`TR001`	상태 전이 위반(종료된 실행 재종료, 보관된 트레일러 생성)	409
`TR002`	같은 대상 실행 진행 중	409
`TR003`	권리 확인 없음	403
`TR004`	선행 단계 미완료(원고·분석·브리프·스토리보드·에셋·승인 조건)	409
`TR005`	플랜 한도(수·포맷·해상도·길이·동시 실행·프리미엄·승인 링크)	403
`TR006`	인용 검증 안 됨 또는 인용 한도 초과	422
`TR007`	엔드카드 규칙 위반	422
`TR008`	대가성 문구 규칙 위반(설명 길이 초과)	422

코드	의미	HTTP
`TR009`	승인 링크 만료·무효	410
`TR010`	FLORA 연동 불가	409
`TR011`	음악 권리 확인 없음 또는 사용할 수 없는 곡	422
`TR409`	장면 동시 편집 충돌	409
`CR001`	크레딧 부족(코어)	402
`AI001`	국외 이전 미동의 상태에서 상용 제공자 요청(코어)	403
`AI002`	국외 이전 동의 필요(코어)	403
42501	권한 없음	403
23503	다른 조직·다른 책 참조	404(존재를 드러내지 않는다)
23514	형식 검사 실패(경로, ISBN, 팔레트, 해시태그 등)	422
P0002	대상 없음	404

8. 로그인 경로 (확정 결정 22)

경로	단계	설명
`POST /api/dev/login`	A(로컬 전용)	시드 페르소나 로그인. `AUTH_MODE=mock`이고 로컬 Supabase일 때만 라우트가 존재 (`AUTH_ROADMAP.md` 2.4절)
`GET /auth/callback`	B, C1	코드 교환, `next` 허용 목록, 첫 로그인 약관 동의(`/onboarding/terms`)
`GET /auth/naver/start`, `/auth/naver/callback`	C2(브리지일 때만)	`AUTH_ROADMAP.md` 7.2절